Hamming (7,4) Error-Correcting Code: Mathematical Basis, Verilog Design and ModelSim Verification

Huynh Do Hoang Phuc Student ID: SS196635
Course: COS201
Instructor: Pham The Vinh

Abstract

Bits can flip when data is sent through a noisy channel or stored in memory. The Hamming (7,4) code adds 3 check bits to every 4 data bits, so the receiver can find and correct any single wrong bit. This report explains the mathematics of the code using simple parity checks (XOR) on bits: how the check bits are chosen, how three checks point to the wrong bit (the syndrome), and the Hamming distance. We show that the minimum distance is 3 and that the code is perfect. We then build an encoder and a decoder in Verilog and test them in ModelSim against a C++ golden model with 128 test vectors (16 data values, each with no error or one flipped bit out of 7). Both modules passed all 128 vectors. On a channel that flips each bit with probability $p = 0.01$, the probability of a wrong 4-bit block falls from 3.94% to 0.20%, about 19 times lower, at the cost of 75% more transmitted bits.

Index Terms

Hamming code, error correction, linear block code, GF(2), syndrome, Verilog, ModelSim.

I. Introduction

A. Problem

Digital data is not always received correctly. Electrical noise or memory faults can turn a 0 into a 1 or the other way around. There are two common ways to deal with this. One is to ask the sender to send the data again, which needs a return channel and takes time. The other is to add extra bits so that the receiver can fix the error by itself. R. W. Hamming proposed a simple code of the second kind in 1950 [1]. Similar codes are still used in ECC memory.

B. Objectives

The objectives of this project are:

1) to explain the mathematics of the Hamming (7,4) code with simple parity checks (XOR);
2) to solve one complete example (encode, add an error, compute the syndrome, correct);
3) to build the encoder and the decoder in Verilog;
4) to verify them in ModelSim against a C++ golden model;
5) to measure how much the code reduces errors.

C. Workflow

Mathematics → C++ golden model → test vectors → Verilog RTL → ModelSim simulation → evaluation

II. Mathematical Background

The results in this section are standard facts about linear block codes. They can be found in [2]–[4].

TABLE I
ADDITION AND MULTIPLICATION IN GF(2).

a	b	$a + b$ (XOR)	$a \cdot b$ (AND)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

TABLE II
BIT LAYOUT OF THE 7-BIT CODEWORD.

Position	1	2	3	4	5	6	7
Binary	001	010	011	100	101	110	111
Bit	p_1	p_2	d_1	p_3	d_2	d_3	d_4

A. The binary field GF(2)

GF(2) has two elements, 0 and 1. Addition is XOR and multiplication is AND (Table I). Every element is its own negative ($a + a = 0$), so adding and subtracting are the same operation. All sums of bits in this report are taken modulo 2.

B. Linear block code (n, k)

A codeword is a binary vector with n bits. An (n, k) code chooses 2^k codewords out of all 2^n vectors, such that the sum of two codewords is always a codeword. For Hamming (7,4):

- $n = 7$ transmitted bits, $k = 4$ data bits, $n - k = 3$ check bits;
- only 16 of the 128 seven-bit vectors are codewords;
- the code rate is $R = k/n = 4/7 \approx 0.571$.

C. Hamming distance and error-correcting power

The *weight* $w(x)$ of a vector is its number of 1s. The *Hamming distance* between two vectors is the number of positions where they differ:

$$d(x, y) = w(x + y). \tag{1}$$

The *minimum distance* $d_{\min}$ of a code is the smallest distance between two different codewords. For a linear code it equals the smallest weight of a non-zero codeword. A code can

- detect up to $d_{\min} - 1$ bit errors, and
- correct up to $t = \lfloor (d_{\min} - 1)/2 \rfloor$ bit errors.

Picture a ball of radius t around each codeword. If the balls do not overlap, a received word lies in at most one ball, and the centre of that ball is the codeword that was sent.

D. Bit layout and check bits

Hamming put the check bits at the positions that are powers of 2. Positions are numbered from 1 to 7 (Table II). Bit 0 of the Verilog vector is position 1.

The 7 positions are looked at in three overlapping groups:

- group 1: positions 1, 3, 5, 7 (binary number ends in 1);
- group 2: positions 2, 3, 6, 7 (binary number has a 1 in the middle);
- group 3: positions 4, 5, 6, 7 (binary number has a 1 on the left).

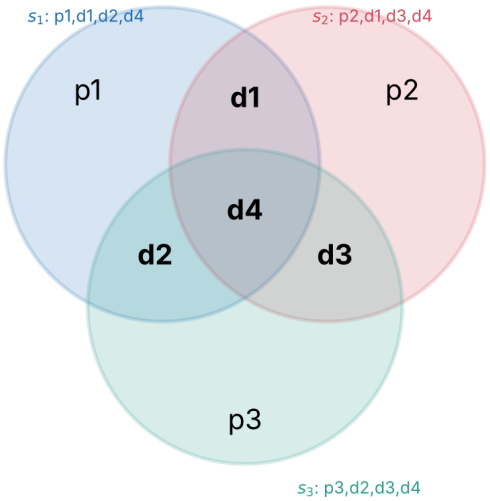


Fig. 1. Each circle is one group (one parity check). Data bits are in the overlaps, and each check bit is in the part of one circle only.

Each group contains exactly one check bit (p_1, p_2 or p_3). The sender chooses the check bits so that each group has an even number of 1s:

$$p_1 = d_1 \oplus d_2 \oplus d_4, \quad p_2 = d_1 \oplus d_3 \oplus d_4, \quad p_3 = d_2 \oplus d_3 \oplus d_4. \tag{2}$$

Fig. 1 draws the three groups as circles.

E. Syndrome and error correction

The receiver gets 7 bits $r_1, \ldots, r_7$ (the bit at position j is r_j) and repeats the three checks:

$$\begin{aligned} s_1 &= r_1 \oplus r_3 \oplus r_5 \oplus r_7, \\ s_2 &= r_2 \oplus r_3 \oplus r_6 \oplus r_7, \\ s_3 &= r_4 \oplus r_5 \oplus r_6 \oplus r_7. \end{aligned} \tag{3}$$

A result of 0 means the group still has an even number of 1s. A result of 1 means the check fails. The three results, read as the binary number $(s_3 s_2 s_1)_2$, are called the *syndrome*.

If no bit is wrong, all three checks pass and the syndrome is 000. If exactly one bit at position j is wrong, then exactly the groups that contain position j fail, because flipping one bit changes the parity of every group that contains it. Every position belongs to a different set of groups, and because of the layout in Table II the failing groups spell out the binary number of j (Table III). So:

- $s = 000$: no error;
- $s \neq 000$: the value of $(s_3 s_2 s_1)_2$ is the *position of the wrong bit* (1 to 7). Flip that bit, then read the data from positions 3, 5, 6 and 7.

This is why the check bits are placed at positions 1, 2 and 4: the syndrome tells the error position directly, with no lookup table.

F. Why 3 check bits for 4 data bits?

With r check bits there are 2^r possible syndromes. They must name each of the $k + r$ positions, plus the case "no error", so we need

$$2^r \;\geq\; k + r + 1. \tag{4}$$

For $k = 4$: $r = 2$ gives $4 \geq 7$, which is false, and $r = 3$ gives $8 \geq 8$, which is true with nothing wasted. So Hamming (7,4) is the smallest single-error-correcting code for 4 data bits.

TABLE III
WHICH CHECKS FAIL WHEN A SINGLE BIT AT EACH POSITION IS WRONG.

Wrong position	Binary	Failing groups	Syndrome $(s_3 s_2 s_1)$
1	001	1	001 = 1
2	010	2	010 = 2
3	011	1, 2	011 = 3
4	100	3	100 = 4
5	101	1, 3	101 = 5
6	110	2, 3	110 = 6
7	111	1, 2, 3	111 = 7
No error	—	none	000 = 0

TABLE IV
ALL 16 CODEWORDS OF HAMMING (7,4), WRITTEN FROM POSITION 1 TO POSITION 7. THE SMALLEST NUMBER OF ONES IN A NON-ZERO CODEWORD IS 3.

Data (binary)	Codeword (positions 1–7)	Ones	Data (binary)	Codeword (positions 1–7)	Ones
0 = 0000	0000000	0	8 = 1000	1101001	4
1 = 0001	1110000	3	9 = 1001	0011001	3
2 = 0010	1001100	3	10 = 1010	0100101	3
3 = 0011	0111100	4	11 = 1011	1010101	4
4 = 0100	0101010	3	12 = 1100	1000011	3
5 = 0101	1011010	4	13 = 1101	0110011	4
6 = 0110	1100110	4	14 = 1110	0001111	4
7 = 0111	0010110	3	15 = 1111	1111111	7

G. The minimum distance is 3

Claim. The Hamming (7,4) code has $d_{\min} = 3$.

Proof. We list all 16 codewords, one for each value of the 4 data bits (Table IV). Adding two codewords bit by bit with XOR always gives another codeword, because every check bit is an XOR of data bits. So the distance between two different codewords is the number of 1s in their sum, which is a non-zero codeword. Therefore $d_{\min}$ is the smallest number of 1s in a non-zero codeword. In the table this number is 3. ■

The weights are: one word with 0 ones, seven with 3, seven with 4 and one with 7 (checked with the attached Python script). With $d_{\min} = 3$ we get $t = 1$: the code corrects exactly one error.

H. Perfect code

A ball of radius 1 around a codeword has $1+\binom{7}{1} = 8$ vectors (the codeword and its 7 one-bit neighbours). The balls do not overlap because $d_{\min} = 3$, so together they cover

$$2^k \left[1 + \binom{n}{1}\right] = 16 \times 8 = 128 = 2^7 \tag{5}$$

vectors, which is all 7-bit vectors. So every received word is either a codeword or exactly one bit away from one codeword. A code that meets the Hamming (sphere-packing) bound with equality is called a *perfect code* [2], [3]. In practice this means the decoder always returns an answer. When two errors happen, the answer is wrong (Section III-C).

I. Performance on a binary symmetric channel

Assume each bit is flipped independently with probability p. Without coding, a 4-bit block is wrong if at least one bit is wrong:

$$P_{\text{uncoded}} = 1 - (1-p)^4 \approx 4p. \tag{6}$$

With Hamming (7,4), the block is decoded correctly when there is *at most one* error among the 7 bits. If there are two or more errors, the decoded word is always a different codeword. The check bits are fixed by the data, so the data is also wrong. Hence

$$P_{\text{Hamming}} = 1 - (1-p)^7 - 7p(1-p)^6 \approx 21p^2, \tag{7}$$

where $21 = \binom{7}{2}$ is the number of ways to choose two wrong bits out of seven. For small p we have $p^2 \ll p$, so the code helps a lot. The price is the code rate $4/7$: 75% more bits are sent.

The data-bit error rate after decoding is computed exactly by listing all 128 error vectors e with probability $p^{w(e)}(1-p)^{7-w(e)}$ and counting the wrong data bits. Because the code is linear, it is enough to send the all-zero codeword.

III. Worked Example

A. Problem

A system sends the data $m = 0101_2 = 5$, that is $d_4d_3d_2d_1 = 0101$, so $d_1 = 1$, $d_2 = 0$, $d_3 = 1$, $d_4 = 0$. The channel flips the bit at **position 5**. Encode the data, compute the syndrome and recover the data.

B. Solution

Step 1: Encode. From (2):

$$p_1 = 1 + 0 + 0 = 1, \quad p_2 = 1 + 1 + 0 = 0, \quad p_3 = 0 + 1 + 0 = 1.$$

The codeword (positions 1 to 7) is $c = (p_1, p_2, d_1, p_3, d_2, d_3, d_4) = (1, 0, 1, 1, 0, 1, 0)$. Written as bits 6 down to 0 in Verilog, this is $0101101_2 = \texttt{2D}_{16}$.

Step 2: The channel flips position 5. $e = (0, 0, 0, 0, 1, 0, 0)$, so $r = c + e = (1, 0, 1, 1, 1, 1, 0)$, which is $0111101_2 = \texttt{3D}_{16}$.

Step 3: Syndrome.

$$\begin{aligned} s_1 &= r_1 + r_3 + r_5 + r_7 = 1 + 1 + 1 + 0 = 1, \\ s_2 &= r_2 + r_3 + r_6 + r_7 = 0 + 1 + 1 + 0 = 0, \\ s_3 &= r_4 + r_5 + r_6 + r_7 = 1 + 1 + 1 + 0 = 1. \end{aligned}$$

So $(s_3 s_2 s_1) = 101_2 = 5$. Checks 1 and 3 fail and check 2 passes, so the wrong bit is in groups 1 and 3 but not in group 2. That is position 5 (Table III).

Step 4: Correct and read the data. Flip r_5 from 1 to 0. We get $(1, 0, 1, 1, 0, 1, 0) = c$. The data bits $(r_3, r_5, r_6, r_7) = (1, 0, 1, 0)$ give $d_1 = 1, d_2 = 0, d_3 = 1, d_4 = 0$, which is the original data $0101_2 = 5$.

This example is the line `5 2D 10 3D 5 5` in `vectors.txt` and the vector under the cursor in Fig. 4.

C. What happens with two errors?

Keep $c = 0101101_2$ but let the channel flip positions 1 and 2, so $r = 0101110_2$. The syndrome is $s = 011_2 = 3$, the same as a single error at position 3 (Table III). The decoder thinks position 3 is wrong and flips it. The output is $(d_1, d_2, d_3, d_4) = (0, 0, 1, 0)$, that is 4 instead of 5. The decoder cannot report this error, because a non-zero syndrome looks like a single error. This is the main limit of Hamming (7,4): it cannot tell one error from two.

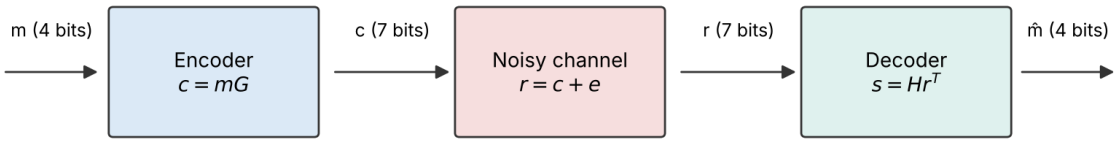


Fig. 2. System block diagram. The encoder adds 3 check bits, the channel flips bits, and the decoder computes the syndrome and corrects one error.

D. A numerical problem

A channel flips each bit with probability $p = 0.01$. We send 10^6 blocks of 4 data bits. From (6) and (7):

- without coding: $1 - 0.99^4 = 0.0394$, about **39 404** wrong blocks;
- with Hamming (7,4): $1 - 0.99^7 - 7(0.01)(0.99)^6 = 0.00203$, about **2 031** wrong blocks.

The number of wrong blocks is about 19 times smaller.

IV. Verilog Design

A. Overview

Both modules are *combinational* (no clock). They use only XOR gates and a small position decoder.

B. Module `hamming_encode`

Input `data[3:0]` (bit 0 is d_1, bit 3 is d_4), output `codeword[6:0]`.

```
assign p1 = data[0] ^ data[1] ^ data[3];
assign p2 = data[0] ^ data[2] ^ data[3];
assign p3 = data[1] ^ data[2] ^ data[3];
assign codeword = {data[3], data[2], data[1],
             p3, data[0], p2, p1};
```

The first three lines are (2). The last line joins the bits from position 7 down to position 1, because Verilog writes the highest bit on the left.

C. Module `hamming_decode`

Input `codeword[6:0]`, outputs `data[3:0]`, `syndrome_out[2:0]` and `error_detected`.

```
wire s1 = codeword[0]^codeword[2]^codeword[4]^codeword[6];
wire s2 = codeword[1]^codeword[2]^codeword[5]^codeword[6];
wire s3 = codeword[3]^codeword[4]^codeword[5]^codeword[6];
assign syndrome_out = {s3, s2, s1};
assign error_detected = (syndrome_out != 3'b000);
wire [6:0] mask = error_detected ?
              (7'b1 << (syndrome_out - 3'd1)) : 7'b0;
wire [6:0] fixed = codeword ^ mask;
assign data = {fixed[6], fixed[5], fixed[4], fixed[2]};
```

TABLE V
LINK BETWEEN THE IDEA AND THE VERILOG CODE.

Idea	Verilog
Choose check bits, Eq. (2)	`p1, p2, p3, codeword` (encoder)
Three checks, Eq. (3)	`s1, s2, s3, syndrome_out`
Mark the wrong position	`mask = 1 << (syndrome - 1)`
Flip the wrong bit	`fixed = codeword ^ mask`
Read positions 7, 6, 5, 3	`data = {fixed[6],fixed[5],fixed[4],fixed[2]}`

D. From the idea to the code

V. EXPERIMENTAL SETUP

A. Golden model

The C++ file `hamming_golden.h` implements the same algorithm with the bit layout of Table II. It has two functions, `hamming_encode` and `hamming_decode`. It is written separately from the Verilog and is used as the reference.

B. Test vectors

The program `gen_vectors.cpp` loops over 16 data values and 8 error cases ($k = 0$: no error; $k = 1$ to 7: flip bit k). This gives 128 vectors in `vectors.txt`. Each line (hexadecimal) is

```
data code_clean err_mask code_rx syndrome data_out
```

C. Testbench

The file `tb_hamming.v` reads each line with `$fscanf`, drives the two modules, and checks:

1) **Encoder:** `enc_code` equals `code_clean` from the golden model.
2) **Decoder:** `dec_data` equals `data_out`, `dec_syn` equals the golden syndrome, and `dec_data` equals the original `data`.
3) `error_detected` is 1 if and only if the syndrome is not zero.

At the end it prints the number of passed and failed vectors and `KET QUA: PASS` or `FAIL`.

D. Simulation environment

ModelSim Intel FPGA Starter Edition 2020.1. The commands, run in the folder that contains `vectors.txt`, are:

```
vlib work
vlog hamming_encode.v hamming_decode.v tb_hamming.v
vsim -voptargs=+acc work.tb_hamming
add wave -radix hex -r /tb_hamming/*
run -all
```

Each vector takes 2 ns (1 ns for the encoder, 1 ns for the decoder), so the 128 vectors end at 256 ns.

E. Independent mathematical check

The Python script `hamming_math_check.py` implements the code a second time, lists the 16 codewords and their weights, computes the exact error rates by listing the 128 error vectors, and runs a Monte Carlo simulation (8×10^5 data bits for each value of p, fixed random seed). It also reads `vectors.txt` and confirms that all 128 lines agree.

```
VSIM 3> run -all
# ================================
#  Tong so vector : 128
#  Encoder  PASS/FAIL: 128 / 0
#  Decoder  PASS/FAIL: 128 / 0
#  KET QUA: PASS
# ================================
# ** Note: $finish    : D:/KY_5/COS201/hamming_code_nop/tb_hamming.v(69)
#    Time: 256 ns  Iteration: 0  Instance: /tb_hamming
# 1
# Break in Module tb_hamming at D:/KY_5/COS201/hamming_code_nop/tb_hamming.v line 69
```

Fig. 3. ModelSim Transcript output of the testbench.

TABLE VI
TESTBENCH RESULTS.

Check	Passed	Failed
Encoder	128	0
Decoder (data, syndrome, original data, error flag)	128	0

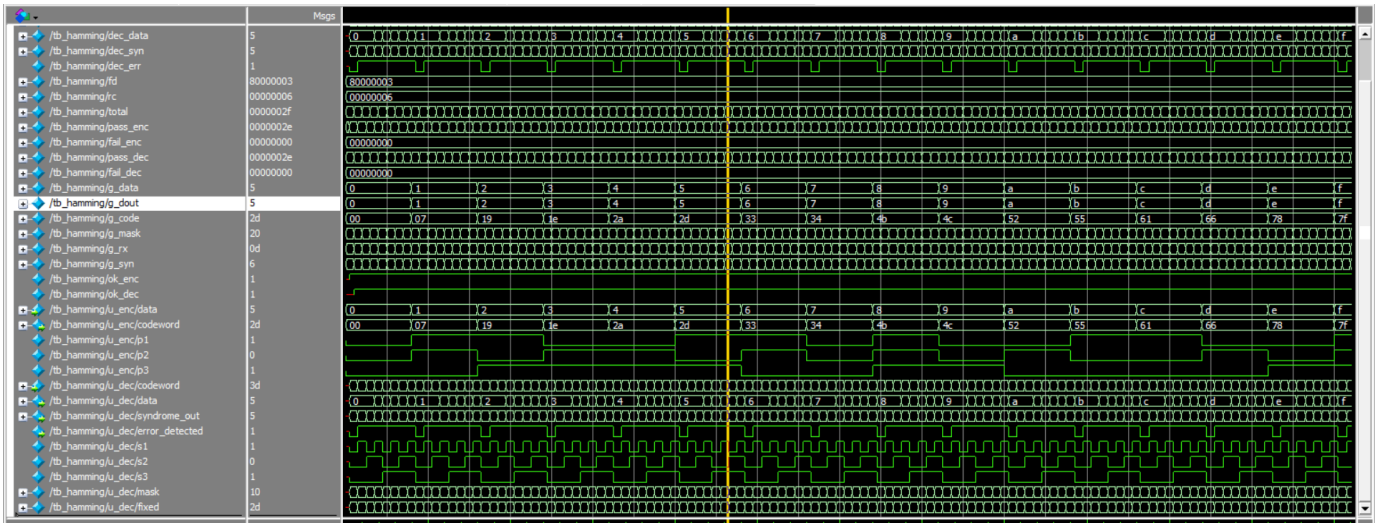


Fig. 4. Simulation waveform. The cursor at 92 929 ps is on the vector `5 2D 10 3D 5 5`: `u_enc/codeword`=2d, `u_dec/codeword`=3d, `s3 s2 s1`=1 0 1, `u_dec/mask`=10, `u_dec/fixed`=2d and `u_dec/data`=5. The signals `g_*` already hold the next line of `vectors.txt`, so they are one step ahead of `u_dec/*`.

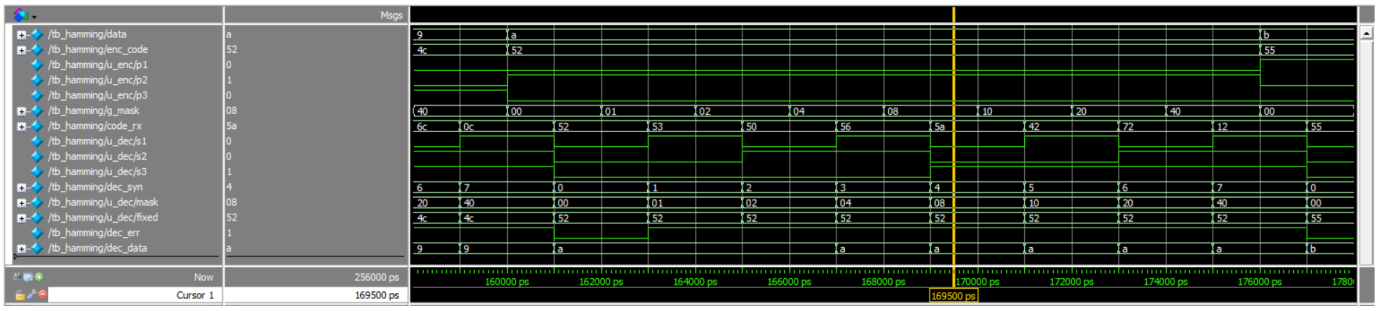


Fig. 5. Zoomed waveform for data `a` (10) from 160 ns to 176 ns. Each 2 ns step flips a different bit (`g_mask` = 00, 01, 02, 04, 08, 10, 20, 40). The syndrome `dec_syn` equals the position of the flipped bit (0 to 7), `dec_err` is 0 only when no bit is flipped, and `dec_data` stays `a`. In the first nanosecond of each step the decoder outputs still belong to the previous vector. The cursor at 169.5 ns shows the case where bit 4 is flipped: syndrome 4, mask 08, fixed 52.

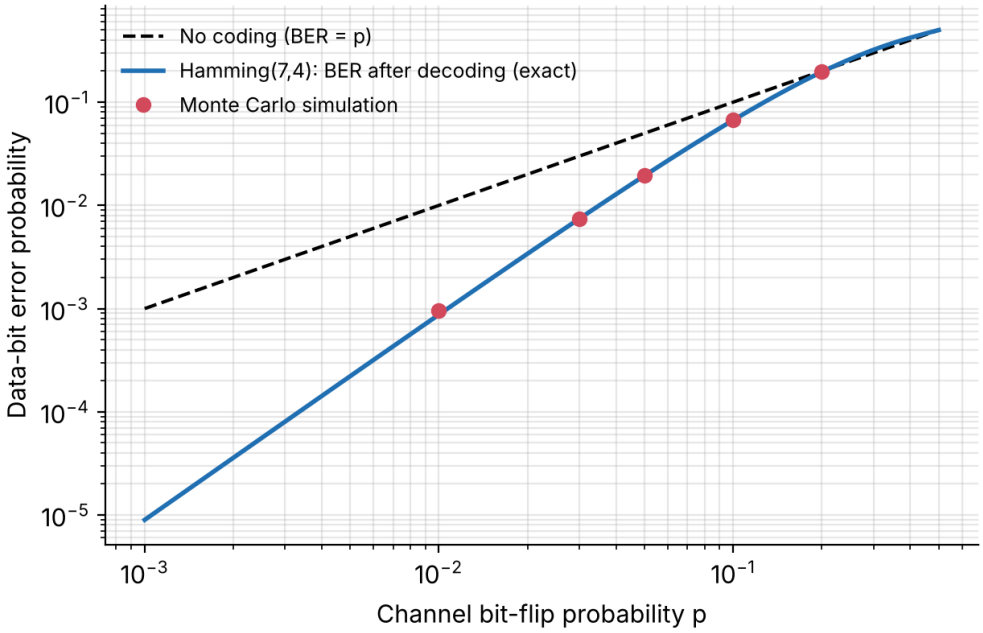


Fig. 6. Data-bit error probability against the channel bit-flip probability p (log scale). The line is the exact result from listing all 128 error vectors. The dots are Monte Carlo results.

TABLE VII
ERROR PROBABILITIES FOR DIFFERENT p.

p	Wrong block, no coding (6)	Wrong block, Hamming (7)	Data-bit error after decoding
0.001	3.99×10^{-3}	2.09×10^{-5}	8.97×10^{-6}
0.01	3.94×10^{-2}	2.03×10^{-3}	8.74×10^{-4}
0.05	1.85×10^{-1}	4.44×10^{-2}	1.94×10^{-2}
0.10	3.44×10^{-1}	1.50×10^{-1}	6.69×10^{-2}
0.20	5.90×10^{-1}	4.23×10^{-1}	1.96×10^{-1}

VI. RESULTS AND DISCUSSION

A. *ModelSim results*

In Fig. 4, the received word `3d` differs from the correct codeword `2d`. The bits `s1, s2, s3` are 1, 0, 1, so the syndrome is 5. The `mask 10` (hex) flips the wrong bit and gives `fixed=2d`, and `dec_data=5` equals the original data. This matches the hand calculation in Section III. The signal `error_detected` is 1 for 7 of every 8 vectors (all cases with a flipped bit), as designed.

B. *Error rate on a binary symmetric channel*

Observations:

- For small p, the data-bit error of Hamming (7,4) has slope 2 on the log–log plot (error $\propto p^2$), while uncoded data has slope 1. At $p = 0.001$ the data-bit error is about $9p^2$.
- The Monte Carlo results agree with the exact values (for example $p = 0.05$: 1.94×10^{-2} and 1.94×10^{-2}; $p = 0.1$: 6.70×10^{-2} and 6.69×10^{-2}). At $p = 0.01$ the simulation gives 9.6×10^{-4} and the exact value is 8.7×10^{-4}. The difference is within the random variation of one run, because bit errors in the same block are correlated.
- The code helps only when p is small enough. Solving "data-bit error after decoding $= p$" gives a limit near $p \approx 0.21$. Above it, two or more errors happen too often and the extra bits do not help.

C. Limitations

- It corrects only one wrong bit. With two or more errors, the decoder can output wrong data without any warning (Section III-C).
- The code rate $4/7$ means 75% more bits. Longer Hamming codes such as (15,11) have a better rate of $11/15$.
- The testbench uses only single-bit errors. It has no 2-error vectors to show the wrong behaviour.
- The error-rate analysis assumes independent bit flips. Burst errors in real systems would give worse results.

VII. Conclusion and Future Work

Conclusion. Hamming (7,4) is a linear block code with $d_{\min} = 3$, and it is a perfect code: it has 16 codewords, each surrounded by a ball of 8 vectors, and $16 \times 8 = 128$ covers the whole space. The syndrome (the result of three parity checks) gives the position of the wrong bit directly, so the decoder needs only a few XOR gates. The Verilog design passed 128 of 128 test vectors against the C++ golden model in ModelSim and agreed with the hand calculation. At $p = 0.01$, the probability of a wrong block is about 19 times lower.

Future work.

1) Describe the same code with matrices over GF(2) (generator and parity-check matrices) and prove its properties formally.
2) Add an overall parity bit to get the extended Hamming (8,4) code (SECDED): correct one error and detect two.
3) Try longer codes such as Hamming (15,11) for a better code rate.
4) Add two-error vectors to the testbench.
5) Synthesize the design in Quartus to get resource use and delay.
6) Make a clocked version with registers for high-speed use.

AI Use Declaration

I used an AI assistant (Claude, by Anthropic) as a helper for this project. It helped me to (1) write the Verilog testbench, (2) suggest the LaTeX structure of this report, and (3) improve the English wording. It also wrote the Python script that checks the mathematics and draws the figures, and it helped me draft the text of this report. I wrote the Verilog encoder and decoder and the C++ golden model myself. I ran all simulations and took all screenshots from my own ModelSim runs. I checked the formulas, the numerical results and the references. I take full responsibility for the content of this report.

References

[1] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, Apr. 1950.
[2] S. Lin and D. J. Costello, *Error Control Coding*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2004.
[3] F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*. Amsterdam, The Netherlands: North-Holland, 1977.
[4] E. W. Weisstein, "Hamming code," MathWorld–A Wolfram Web Resource, https://mathworld.wolfram.com/HammingCode.html.